

P14 Report engine: checks, drill down, Bill List and export

Model: Opus, high effort.

Branch: none. Commit directly to `main`.

Depends on: P13.

1. What to build, in one sentence

Build the one engine every M19 report runs through, complete the balance checks, build R19 Bill List as the target of every drill down, add Excel export, and add the indexes P13 listed, so P15 to P17 only have to define each report's grouping.

2. Module

M19 Reports v2.

3. The spec is already written

`docs/API-CONTRACT.md` section "M19 Reports v2", written by P13.

Build exactly what it says. If the spec and this prompt disagree, the spec wins, and you tell me.

If the spec is silent on something you need, stop and ask.

4. Step 0. Save this prompt and check the repo

1. Save this entire prompt, exactly as given, to `docs/prompts/P14-report-engine.md`.
2. `git pull` . `git status` should show only that file.
3. `docs/prompts/README.md` must show P13 as Done. If not, stop.
4. Run `npm test` and record the count. It should match P12's "after" count, since P13 changed no code. If anything fails, stop and tell me.

5. Files to read first

1. The M19 section of `docs/API-CONTRACT.md` , all of it.
 2. `docs/RECONCILIATION-RULES.md` , all of it.
 3. `docs/TEST-DATA.md` section 6, breaking the rules on purpose.
 4. `server/services/reportRangeService.js` ,
`server/services/salesReportService.js` ,
`server/services/dayFiguresService.js` ,
`server/services/reconciliationService.js` .
 5. `server/tests/helpers/goldenDay.js` and
`server/tests/goldenDay.test.js` from P10.
 6. `server/routes/reportRoutes.js` and
`server/validators/reportValidators.js` .
-

6. Part A. The engine

Create `server/services/reports/` with:

`engine.js`, exporting `runReport(req, definition, params)`. In this order, and nowhere else:

1. Validate `params` against the definition's schema.
2. Check the range against `MAX_RANGE_DAYS` with the existing `assertRange`.
3. Build the base match: tenant scope, branch, the business date range on the right field, and voided bills left out. Reuse `tenantMatch` and `liveInRange` from `reportRangeService.js`. A definition never builds its own tenant match.
4. Call the definition's `query(req, baseMatch, params)`, which returns rows and totals in paise.
5. Compute `openDays` from `dayclosures`.
6. Run the definition's checks through `reconciliationService.js`.
7. Build the `filterSentence` from the params, the business day start setting, and the filters' plain names.
8. Attach `columns` with their glossary labels, and drill downs, exactly as the contract shapes them.
9. Return the envelope.

`definitions/`, one file per report. This prompt adds `bills.js` for R19 and one small definition used only by tests. P15 to P17 add the rest.

`registry.js`, mapping report ids to definitions, and a test that every registered definition has a title, roles, a schema,

columns with labels that exist in a glossary label list, and checks.

`labels.js`, the glossary labels as constants, so no definition types a label by hand. Mirror it on the client with no imports, and test that the two match.

One route file, `server/routes/reportV2Routes.js`, mounting each registered report at its contract path with its roles. Existing M6 routes stay where they are, untouched.

Rules every definition must follow, written into a comment at the top of `engine.js`:

sums in paise only, through `$sum` or `sumPaise`;
averages as a sum divided by a sum, rounded with the `money.js` rule, only after totalling;
group only on frozen fields;
never read `menuitems`, `categories`, `users` or `paymentmethods` to produce a figure.

7. Part B. The rest of the checks

Complete `server/services/reconciliationService.js` with every check in `docs/RECONCILIATION-RULES.md`:

1. Add C2, C5, C7, C10, C11 and C12.
2. Add range versions of C1, C3, C4, C6 and C8, so a report over a month checks every day in it. A range check returns one result, listing every failing day or bill in `refs`.
3. C5 takes the grouped rows and the ungrouped total from the definition, so each report checks its own grouping against the whole.

4. C12 compares each closed day's stored `dayclosures.snapshot` with a fresh `computeDayFigures` for that date.
5. Every message uses the exact wording in `docs/RECONCILIATION-RULES.md`.
6. A check never changes data, and never throws for a failed rule. It returns a failed result.

Wire Day Close to use the completed service, so closing a day now runs every check that applies to one day.

8. Part C. R19 Bill List

Build R19 as the first real definition, exactly as the contract specifies its filters and columns.

It is also the page every drill down opens, so it must accept every filter the contract lists.

Paging: use the repo's existing paging convention from `docs/CONVENTIONS.md` section 3. The totals row covers every matching bill, not just the page.

`GET /api/v1/reports/bills/:billId` returns one bill in full: lines with shares, payments with corrections, discount, account charge, void details, and a timeline of the order's events from opening to payment, read from the order and the bill.

9. Part D. Excel export

1. Add `exceljs` to the server.

2. `server/services/reports/exportXlsx.js` turns any envelope into a workbook, exactly as the contract's export section says: four sheets, money as rupee numbers with an Indian number format, the file name rule.
3. The engine calls it when `format=xlsx`, using the same envelope it built, never a second query.
4. The response is a file download with the right content type and file name.
5. An export where an ERROR check failed still downloads, and its first sheet starts with a line saying which check failed. R9 is the exception, as the spec says: it refuses to build.

10. Part E. Indexes

Add every index P13 listed, in the models, so `npm run db:indexes` builds them on deploy.

Each starts with `restaurantId`.

Run `npm run db:indexes` locally twice, and confirm the second run creates nothing.

11. Part F. The client

Only what drill down needs. The report screens themselves come in P18.

1. A Bill List page at `/reports/bills`, reading its filters from the address, so any drill down link opens it already filtered.

2. Its filter sentence at the top, the check strip under it, the table, paging, and the totals row.
3. Clicking a bill opens the bill detail with its timeline.
4. An "Excel" button that downloads the same filtered list.

Follow `docs/DESIGN-SYSTEM.md` . Use the labels mirror, never typed labels.

12. Tests

Engine

1. A guard test reads every file in `server/services/reports/definitions/` and fails if any imports the `MenuItem` , `Category` , `User` or `PaymentMethod` model. Same idea as the P01 time display guard.
2. A report for restaurant A never includes restaurant B's bills, using two golden day restaurants.
3. The filter sentence reads exactly as the contract's example for the golden day.
4. `openDays` lists 26 September before Day Close and is empty after.
5. A range over `MAX_RANGE_DAYS` is refused.
6. Every registered definition passes the registry test, and the labels mirror test passes.

Checks

7. On the golden day, every check passes except the C9 warning.

8. For every row of `docs/TEST-DATA.md` section 6, break exactly that one thing in the stored documents and confirm exactly that check fails, with its message and the right `refs`. These are the most important tests in this prompt.
9. C12: close 26 September, then change B06's stored bill total directly, and C12 fails for that date and names it.
10. A range check over 26 and 27 September reports failures from both days in one result.

R19

11. Each filter on its own returns exactly the right golden day bills.
12. Combined filters intersect correctly.
13. The totals row covers every matching bill, across pages.
14. Bill detail for B13 shows its two cancelled items in the timeline, and B14's payment at 12:02 AM on 27 September with business date 26 September.

Export

15. Export R19 for the golden day, open it again with `exceljs` in the test, and check the four sheets, the money values in rupees, and that the totals equal the JSON response.

Speed, opt-in

16. Behind `PERF=1`, insert a year of synthetic bills, about 66,000, the size of Caffeza's year, directly into the test database, and confirm R19 with a full-year range and its totals answers in under two seconds. Print the timing. Skip when `PERF` is unset, so the normal suite stays fast.

Run the full suite at the end. Every test that passed before must still pass.

13. Non-negotiable rules that apply

"Every database record has a `restaurantId` . Every query filters by it." The engine owns the tenant match.

"Reports only add up values frozen onto records when the event happened."

"Every report goes through the one shared report engine."

"Every label on a screen or an export comes from `docs/GLOSSARY.md` ."

"A totals row is the exact sum of its rows. An average is a sum divided by a sum."

"Every report runs its checks. A new check gets a test that breaks it on purpose."

14. Checks and golden day

Every check in `docs/RECONCILIATION-RULES.md` now exists, with a test that breaks it on purpose.

The golden day acceptance test from P10 must still pass, now with every one-day check running at close.

15. Docs to update

1. `docs/RECONCILIATION-RULES.md` section 1: every check is built, in `reconciliationService.js` , completed in P14.
2. `docs/PROJECT-STATE.md` :
 1. The date line.

2. "Current stage": "Next: P15, daily, money and GST reports."
 3. Every detail where the build differed from the M19 spec, as a decision row, and the spec updated to match.
 4. Record the opt-in speed test's timing in the P14 entry.
 5. "What changed recently": a P14 entry at the top, and the oldest moved to the archive.
3. `docs/prompts/README.md` : mark P14 as Done.
-

16. Out of scope

Every report except R19. They come in P15, P16 and P17. The report screens, charts and the Today screen. That is P18.

Removing or changing any M6 endpoint.

PDF generation on the server.

17. Done when

1. `npm test` passes, with before and after counts recorded, and `PERF=1 npm test` has been run once with its timing recorded.
2. `npm run lint` and `npm run build` pass.
3. `npm run db:indexes` creates the new indexes, and a second run creates nothing.

4. By hand: open a bill list filtered by a captain through its address, open one bill's timeline, and download the Excel file and open it.
5. Every doc in section 15 is updated.
6. Commits on `main` , one line each, for example:
 - `add report engine and registry`
 - `complete reconciliation checks`
 - `add bill list report`
 - `add excel export`
 - `add report indexes`
 - `update docs for p14`
7. Push `main` .
8. Print a short summary: commits, files changed, test counts before and after, the speed timing, and every place the build differed from the spec.